Submission Worksheet

Submission Data

Course: IT114-007-F2025
Assignment: IT114 Milestone 1
Student: Evans M. (ekn3)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 10/31/2025 12:47:12 PM
Updated: 11/3/2025 10:53:13 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/grading/ekn3
View Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/view/ekn3

Instructions

- Overview Link: https://youtu.be/9dZPFwi76ak

1. Refer to Milestone1 of any of these docs:
    2. Rock Paper Scissors
    3. Basic Battleship
    4. Hangman / Word guess
    5. Trivia
    6. Go Fish
    7. Pictionary / Drawing
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy `Part5` and rename the copy as `Project` (this new folder should be in the root of your repo)
6. Organize the files into their respective packages `Client`, `Common`, `Server`, `Exceptions`
    1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
    1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `main`
10. Upload the same PDF to Canvas
11. Sync Local

1. git checkout main
2. git pull origin main

Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections

```
Evans Mensah@FIB-Surveillance MINGW64 ~/Downloads/IT 114/New folder/Ekn3-IT114-07 (main)
$ java Projects.Server
Server Starting
Server: Listening on port 3000
Room[lobby]: Created
Server: Created new Room lobby
Server: Waiting for next client
```

listening server's screenshot

```
//ekn4 10-41-24      You, 1 second ago • Uncommitted changes
try (ServerSocket serverSocket = new ServerSocket(port)) {
    createRoom(Room.LOBBY);// create the first room (lobby)
    while (isRunning) {
        info(message: "Waiting for next client");
        Socket incomingClient = serverSocket.accept(); // blocking action, waits for a client connection
        info(message: "Client connected");
        // wrap socket in a ServerThread, pass a callback to notify the Server when
        // they're initialized
        ServerThread serverThread = new ServerThread(incomingClient, this::onServerThreadInitialized);
        // start the thread (typically an external entity manages the lifecycle and we
        // don't have the thread start itself)
        serverThread.start();
        // note: we don't yet add the ServerThread reference to our connectedClients map
    } // #58-68 while (isRunning)
} catch (DuplicateRoomException e) {
    System.out.println(Textfx.colorize(text: "Lobby already exists (this shouldn't happen)", Color.RED));
} catch (IOException e) {
    System.err.println(Textfx.colorize(text: "Error accepting connection", Color.RED));
    e.printStackTrace();
} finally {
    info(message: "closing server socket");
}
// #44-72 private void start(int port)
```

server code

Saved: 10/31/2025 1:36:35 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

It creates a new room on a specific port and uses the serverthread to handle all communications. It

Saved: 10/31/2025 1:36:35 PM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

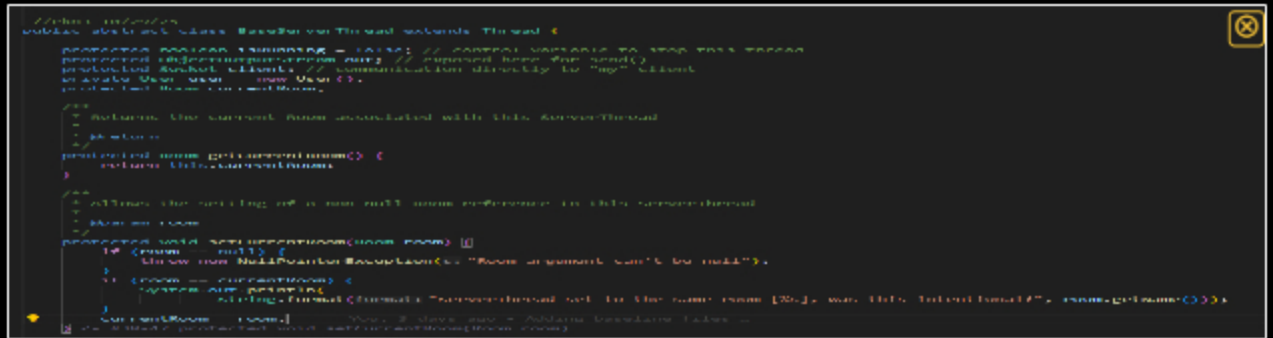
Progress: 100%

Details:

- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections



Server and Client Connection



Code snippet from BaseServerThread

```
//ekn1 10/29/25       You, 1 second ago • Uncommitted changes
protected boolean sendToClient(Payload payload) {
    if (!isRunning) {
        return true;
    }
    try {
        info("Sending to client: " + payload);
        out.writeObject(payload);
        out.flush();
        return true;
    } catch (IOException e) {
        info(message: "Error sending message to client (most likely disconnected)");
        // comment this out to inspect the stack trace
        // e.printStackTrace();
        cleanup();
        return false;
    } <- #128-126 catch (IOException e)
```

code snippet

Saved: 11/3/2025 8:12:42 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients

Your Response:

It allows all clients to send and receive messages without blocking the server or the clients

Saved: 11/3/2025 8:12:42 PM

Section #3: (2 pts.) Feature: Server Will Implement The Concept Of Rooms (With The Default Being "l obby")

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

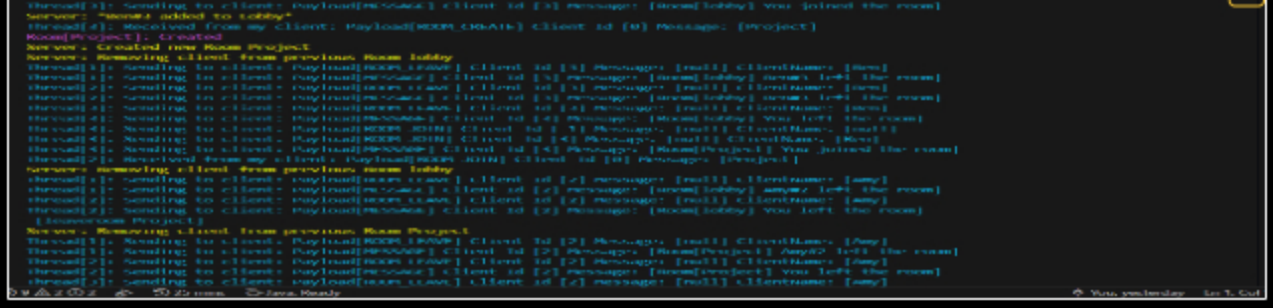
Part 1:

Progress: 100%

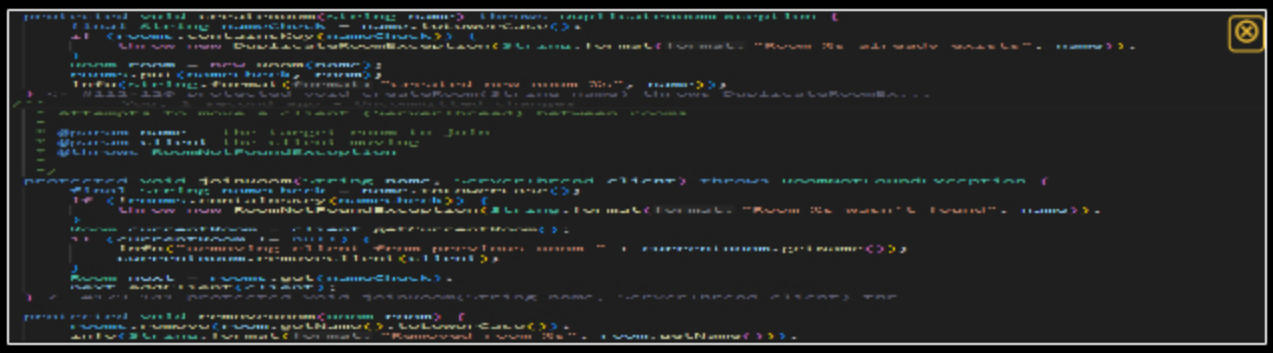
Details:

- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

```
kyan Pendaglia-Surveillance MINGW64 ~/Downloads/1! 114/New folder/ekn1-1!114-0/ (main)
$ java Project4.Server
```



Terminal-Output



code snippet

Saved: 11/3/2025 8:51:34 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

the server waits for the commands /creatroom and checks to see if there aint no room of the same name if not then create one and calls joinroom to place the clients to the new room. If a clients wants to join a room, the server checks for the name and uses addclient method to add them. With the remove client, the server calls the room.removeclient method and take the client out. if the room is empty, its cleans and parmently deletes the room

Saved: 11/3/2025 8:51:34 PM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

Task #1 (1 pt.) - Evidence

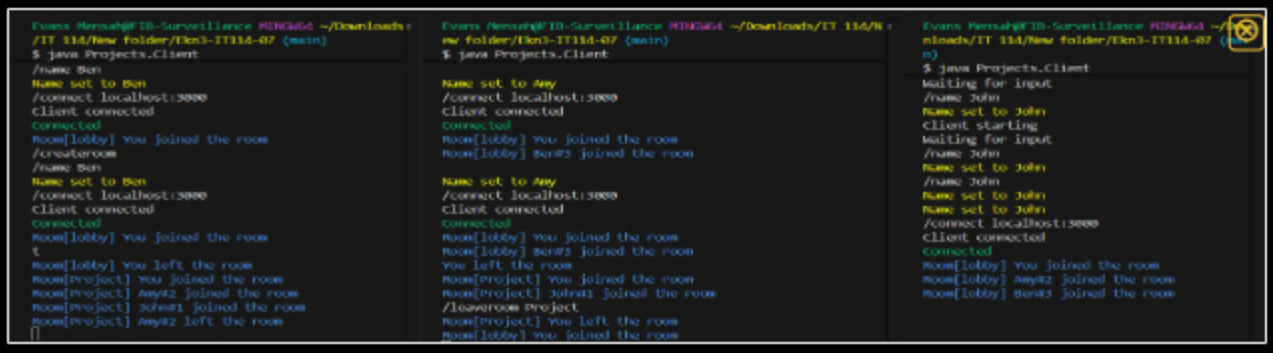
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected



output of succesful connection with commands

```
// End Send*() Methods
//ckn3 10/30/25       You, 1 second ago • Uncommitted changes
@Override
protected void processPayload(Payload incoming) {

    switch (incoming.getPayloadType()) {
        case CLIENT_CONNECT:
            setClientName(((ConnectionPayload) incoming).getClientName().trim());

            break;
        case DISCONNECT:
            currentRoom.handleDisconnect(this);
            break;
        case MESSAGE:
            currentRoom.handleMessage(this, incoming.getMessage());
            break;
```

code snippet

```
            break;
        } <- #140-166 switch (incoming.getPayloadType())
    } <- #138-167 protected void processPayload(Payload incoming)

    @Override
    protected void onInitialized() {
        // once receiving the desired client name the object is ready
        onInitializationComplete.accept(this);
    }
        You, 3 days ago • Adding baseline files  <- #18-174 public class ServerThread extend
```

confirmation

```
//ckn3 10/30/25
private void onServerThreadInitialized(ServerThread serverThread) {
    // Generate server-controlled clientId
    nextClientId = Math.max(nextClientId, b: 1);
    serverThread.setClientId(nextClientId);
    serverThread.sendClientId();// syncs the data to the client
    // add initialized client to the lobby
    info(String.format(format: "*%s initialized*", serverThread.getDisplayName()));
    try {
        joinRoom(Room.LOBBY, serverThread);
        info(String.format(format: "*%s added to Lobby*", serverThread.getDisplayName()));
    } catch (RoomNotFoundException e) {
        info(String.format(format: "*Error adding %s to Lobby*", serverThread.getDisplayName()));
        e.printStackTrace();
```

```
} <-  #90-104 private void onServerThreadInitialized(ServerThread serverThr...
```

confirmation

```
 */
//ekn3 10/30/25      You, 1 second ago • Uncommitted changes
protected boolean sendClientId() {
    ConnectionPayload payload = new ConnectionPayload();
    payload.setPayloadType(PayloadType.CLIENT_ID);
    payload.setClientId(getClientId());
    payload.setClientName(getClientName());// Can be used as a Server-side override of username (i.e., profanity
                                           // filter)
    return sendToClient(payload);
} <- #112-119 protected boolean sendClientId()

/**
```

confirmation

Saved: 11/3/2025 9:05:35 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

the client-server communicates on payload using payload type then the server manages the state for each client and forwards it to the room created or joined or to the lobby

Saved: 11/3/2025 9:05:35 PM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

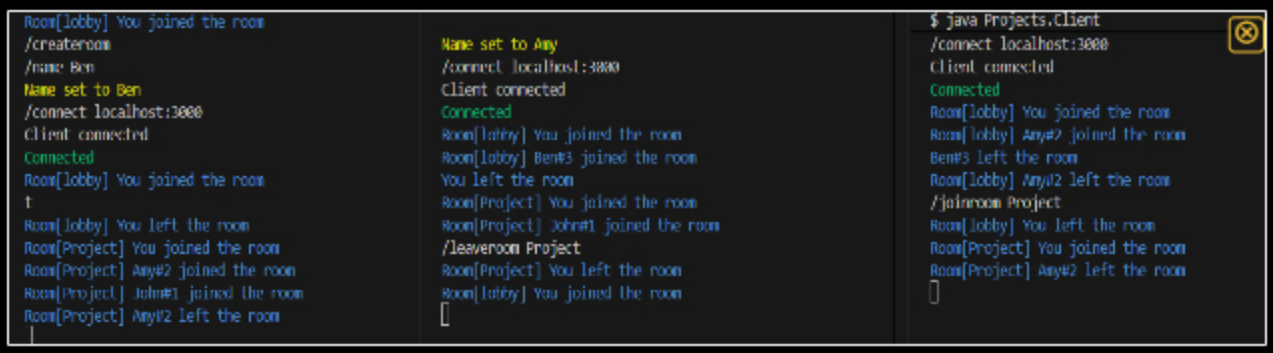
Part 1:

Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining

and joining



Evidence of succesful creation of /joinroom and /createroom

```
//ekn1 10/30/25        You, 1 second ago • Uncommitted changes
private void sendRoomAction(String roomName, RoomAction roomAction) throws IOException {
    Payload payload = new Payload();
    payload.setMessage(roomName);
    switch (roomAction) {
        case RoomAction.CREATE:
            payload.setPayloadType(PayloadType.ROOM_CREATE);
            break;
        case RoomAction.JOIN:
            payload.setPayloadType(PayloadType.ROOM_JOIN);
            break;
        case RoomAction.LEAVE:
            payload.setPayloadType(PayloadType.ROOM_LEAVE);
            break;
        default:
            System.out.println(TextFX.colorize(text: "Invalid room action", Color.RED));
            break;
    } <  #202 215 switch (roomAction)
    sendToServer(payload);
```

code snippet

```
//ekn1 11/02/25        You, 1 second ago • Uncommitted changes
protected boolean sendToClient(Payload payload) {
    if (!isRunning) {
        return true;
    }
    try {
        info("Sending to client: " + payload);
        out.writeObject(payload);
        out.flush();
        return true;
    } catch (IOException e) {
        info(message: "Error sending message to client (most likely disconnected)");
        // comment this out to inspect the stack trace
        // e.printStackTrace();
        cleanup();
        return false;
    } <  #122 128 catch (IOException e)
} <  #113 129 protected boolean sendToClient(Payload payload)
```

code snippet

Saved: 11/3/2025 10:06:12 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /createroom and /join room commands work and the related code flow for each

Your Response:

it translate the users intention to the correct payload type and deliver the that payload to the server

Saved: 11/3/2025 10:06:12 PM

Section #6: (1 pt.) Feature: Client Can Send Messages

Progress: 100%

Task #1 (1 pt.) - Evidence

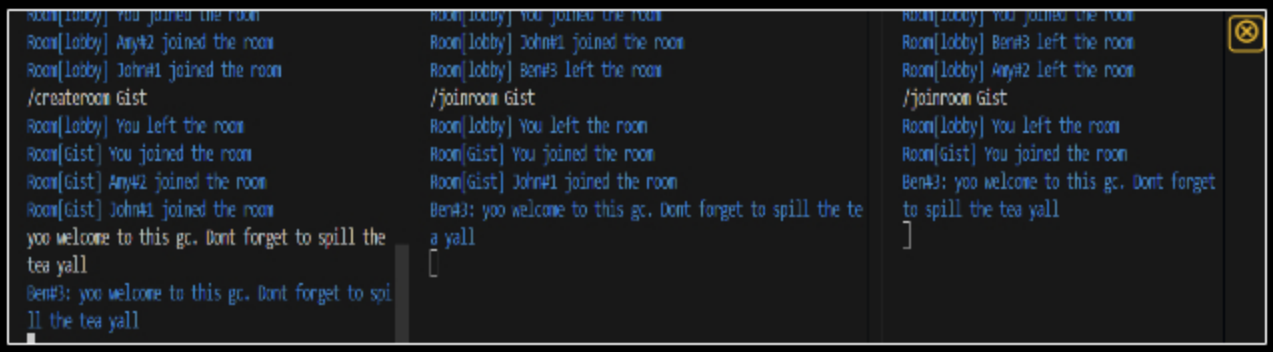
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of a few messages from each of 3 clients
- Include examples of clients grouped into other rooms
- Show the relevant snippets of code that handle the message process from client to server-side and back



output

```java
@Override
protected void processPayload(Payload incoming) {

    switch (incoming.getPayloadType()) {
        case CLIENT_CONNECT:
            setClientName(((ConnectionPayload) incoming).getClientName().trim());

            break;
        case DISCONNECT:
            currentRoom.handleDisconnect(this);
            break;
        case MESSAGE:
            currentRoom.handleMessage(this, incoming.getMessage());
            break;
```

code snippet

```java
//ekn3 11/2/25      You, 1 second ago • Uncommitted changes
    protected synchronized void handleMessage(ServerThread sender, String text) {
        relay(sender, text);
    }
    // end handle methods
} <- #12-268 public class Room implements AutoCloseable
```

code



code

```
//ekn3 11/02/25      You, 1 second ago • Uncommitted changes
protected boolean sendMessage(long clientId, String message) {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.MESSAGE);
    payload.setMessage(message);
    payload.setClientId(clientId);
    return sendToClient(payload);
} <- #129-135 protected boolean sendMessage(long clientId, String message)
```

code

Saved: 11/3/2025 10:20:08 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

The clients sends payload, thread receives and delegates request and sends to all. Server threads delivers the new payload back to the client

Saved: 11/3/2025 10:20:08 PM

Section #7: (1 pt.) Feature: Disconnection

Progress: 100%

Task #1 (1 pt.) - Evidence

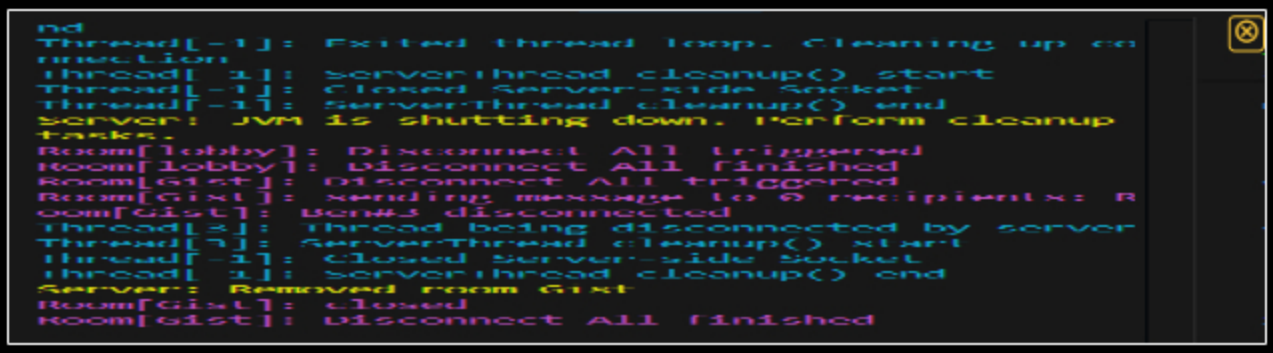
Progress: 100%

Part 1:

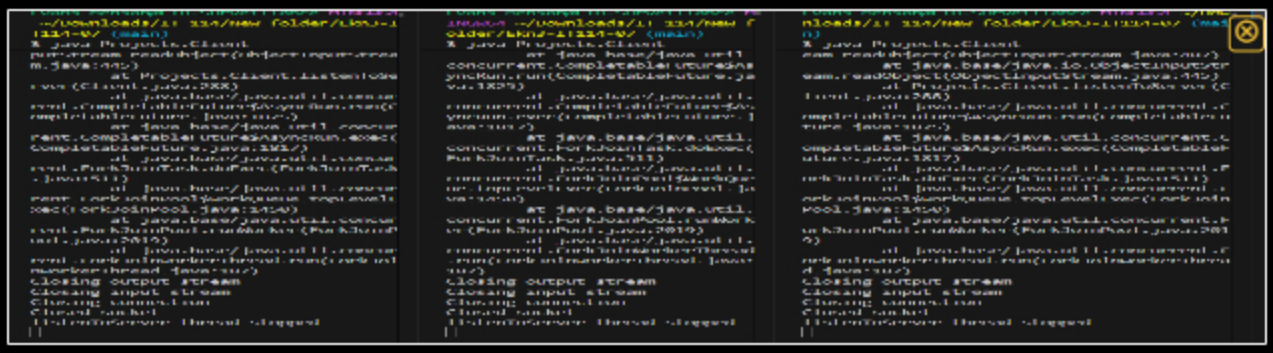
Progress: 100%

Details:

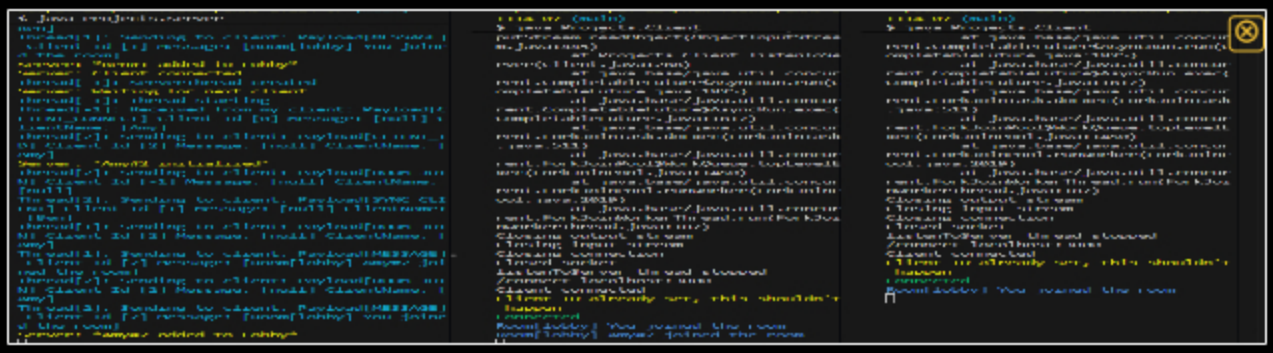
- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occuring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process



Server disconnecting



clients disconnecting



Clients reconnecting

```
//ekn3 11/03/25
private void sendDisconnect() throws IOException {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.DISCONNECT);
    sendToServer(payload);
```

```
} <- #239-243 private void sendDisconnect() throws IOException
```

code snippet of client side disconnection process

```
} finally {
    if (currentRoom != null) {
        currentRoom.handleDisconnect(this);
    }
    isRunning = false;
    info(message: "Exited thread loop. Cleaning up connection");
    cleanup();
} <- #194-201 finally
<- #144-202 public void run()
```

code snippet server-side termination process

```
//ckh3 11/02/25
protected boolean sendToClient(Payload payload) {
    if (!isRunning) {
        return true;
    }       You, 4 days ago • Adding baseline files
    try {
        info("Sending to client: " + payload);
        out.writeObject(payload);
        out.flush();
        return true;
    } catch (IOException e) {
        info(message: "Error sending message to client (most likely disconnected)");
        // comment this out to inspect the stack trace
        // e.printStackTrace();
        cleanup();
        return false;
    } <- #122-128 catch (IOException e)
} <- #113-129 protected boolean sendToClient(Payload payload)
```

code snippet of client side disconnection process

```
protected void cleanup() {
    info(message: "ServerThread cleanup() start");
    try {
        // close server-side end of connection
        currentRoom = null;
        out.close();        You, 4 days ago • Adding baseline files
        client.close();
        user.reset();
        info(message: "Closed Server-side Socket");
    } catch (IOException e) {
        info(message: "Client already closed");
    }

    info(message: "ServerThread cleanup() end");
} <- #207-221 protected void cleanup()
```

code snippet server-side termination process

Saved: 11/3/2025 10:44:50 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

Both the client and server use specific logic to handle disconnects ensuring that the state remains clean

Saved: 11/3/2025 10:44:50 PM

Section #8: (1 pt.) Misc

Progress: 100%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

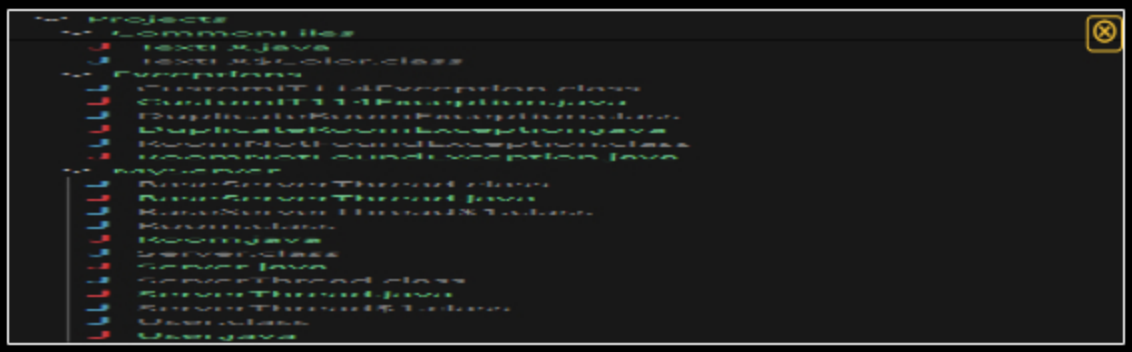
Progress: 100%



structure



Structure



continuation

Saved: 11/1/2025 7:02:19 PM

Task #2 (0.25 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



pull request

Saved: 11/3/2025 10:51:11 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/Faculty06/Ekn3-
IT1 [illegible]

https://github.com/Faculty06/Ekn

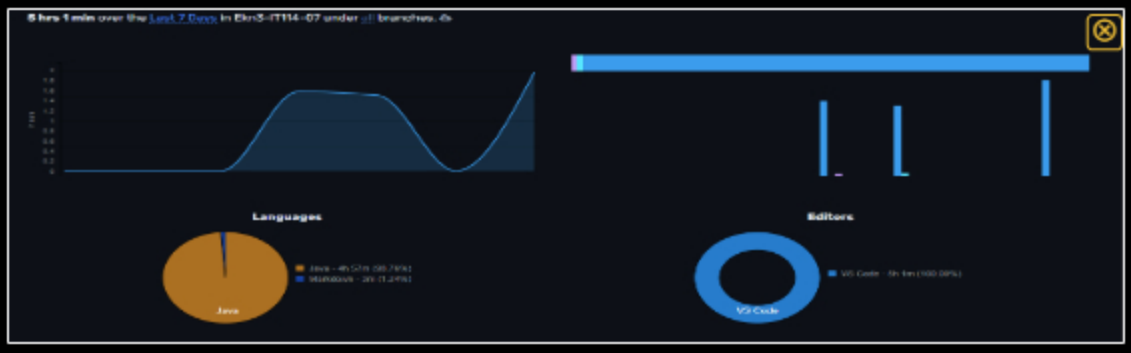
Saved: 11/3/2025 10:51:11 PM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waka time



waka time

Saved: 11/3/2025 10:53:13 PM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

i learned how to properly interconnect packages into folders and how misplacing a package in a folder can cause a huge headache.

Saved: 11/1/2025 7:38:28 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

testing the outputs from the terminal

Saved: 11/1/2025 7:39:09 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Finding the corresponding codes since they are interconnected and have to refer to different packages to track them.

Saved: 11/3/2025 6:02:08 PM